

基于Apache Arrow与CUDA内存模式的TPC-H Q5查询实验

徐子桓 2025104082

摘要 本文围绕内存数据库中的CPU-GPU协同查询处理，实现了一个固定的TPC-H Q5物理执行器。实验保留Q5需要的六张表和字段，使用Apache Arrow IPC作为可复现的数据交换格式，并比较Arrow CPU算子、专用CPU实现、三种CUDA内存模式、CPU-GPU混合执行以及cuDF算子库。三种CUDA模式分别是显式PCIe复制、managed memory预取和mapped pinned host memory直接访问。所有正式后端从同一Arrow数据集读取，先用tiny数据检查查询逻辑，再在官方TPC-H SF1上完成19组配置、57次预热和190次正式测量。结果哈希均为542abf4003633c7c，并逐行通过官方答案核对。专用CPU在16线程时的查询中位数为61.414 ms，cuDF为116.427 ms，三种CUDA模式为314.151至412.264 ms。混合执行虽然正确并有并发时段，但最佳中位数仍为222.832 ms，没有超过专用CPU。当前实现没有完整SQL解析和代价优化，GPU也只负责最后的lineitem扫描，因此结论只适用于本项目的固定查询、冷进程和SF1规模。

关键词 内存数据库；TPC-H Q5；Apache Arrow；CUDA；统一虚拟寻址；cuDF

Abstract This report studies CPU-GPU query processing with a fixed implementation of TPC-H Query 5. Apache Arrow IPC is used as the reproducible interchange format. The experiment compares an Arrow CPU baseline, a specialized CPU executor, three CUDA memory modes, a concurrent CPU-GPU path, and a cuDF operator baseline. Nineteen SF1 configurations completed 190 measured cold-process runs with the same official result. The best specialized CPU median was 61.414 ms. The best hybrid median was 222.832 ms, so the hybrid-speedup hypothesis was rejected for this implementation. The prototype does not implement a general SQL optimizer, and the GPU only scans and aggregates the lineitem table. The conclusions are therefore limited to this query, hardware, execution lifecycle, and SF1 data set.

Keywords in-memory database; TPC-H Q5; Apache Arrow; CUDA; UVA; cuDF

1 引言

GPU具有较高的并行计算能力和显存带宽，但数据库查询不只包含计算，还包含数据装载、连接准备、主机与设备之间的数据移动等步骤。只比较一个CUDA核函数容易得到不完整的结论。因此，本实验选用TPC-H Q5，把CPU列式算子、专用CPU执行器、不同GPU内存模式和GPU算子库放在同一查询语义下进行对比。

本项目不是通用数据库。它没有SQL解析器、逻辑优化器和事务模块，而是把Q5的连接顺序和过滤过程直接写成物理执行计划。这样实现范围较小，适合课程实验，但也意味着结果不能直接推广到其他查询。

2 查询与数据表示

Q5包含六张表。region和nation用于确定地区与国家，customer和orders用于确定客户与订单，supplier和lineitem用于供应商匹配和收入计

算。查询选择指定地区及一年日期范围内的订单，要求客户与供应商属于同一国家，最后按国家聚合

$$\text{revenue} = \sum l.\text{extendedprice} \times (1 - l.\text{discount}). \quad (1)$$

项目只保留查询需要的列。字符串使用字典编码，日期转换为从Unix epoch开始的天数。价格以分为单位保存，折扣以百分之一为单位保存。单行收入用整数表示为

$$r_{10^4} = p_{cent} \times (100 - d_{0.01}), \quad (2)$$

先按 10^4 尺度求和，输出时再舍入到两位小数。这一处理避免原版本逐行截断不足一部分的问题。

数据准备脚本把TPC-H的六个.tbl文件转换为Arrow IPC文件，同时写入schema、行数、record batch和SHA256校验信息。正式实验中，专用C++、Acero、三种CUDA、混合执行和cuDF都读取这份Arrow数据集。它们的输入格式相同，但内部准备并不相同：专用路径会构造按键直接索引的数组，cuDF会把Arrow表转换成DataFrame。因此统一Arrow解决了输入可比性问题，没有消

除执行器准备开销。

3 查询执行方法

3.1 CPU过滤传播

执行器先在CPU端找到目标region中的nation，然后构造supplier、customer和order到nation的三个直接索引数组，以下分别记为 $S[s]$ 、 $C[c]$ 和 $O[o]$ 。日期或地区条件不满足的位置写为-1。最后扫描lineitem，只要订单国家与供应商国家相同，就把收入加入对应国家。该方法避免物化六表连接，但依赖TPC-H键值比较稠密的特点。

专用CPU版本按Arrow record batch边界分片，并把lineitem划分为连续区间。每个线程使用本地国家聚合数组，结束后再归并。另一个CPU版本使用Arrow Acero的filter、hash join、aggregate和order by节点，作为通用列式算子计划对照。二者处理相同语义，但专用版本利用了固定Q5和稠密键，不能代表通用SQL执行器都能采用这种结构。

3.2 CUDA内存模式

三个手写CUDA后端复用CPU生成的过滤传播数组，并执行同一个lineitem聚合核：

- gpu-copy使用device buffer和显式复制；
- gpu-managed使用统一内存并在执行前预取；
- gpu-mapped使用映射锁页内存，GPU经PCIe读取主机数据。

这三个模式比较的是输入数组到GPU的存放和访问方式，并不是三种不同的连接算法。managed提供统一虚拟地址，但数据仍要迁移；mapped省去大块显式H2D，GPU却要经PCIe读取主机页。cuDF版本通过通用GPU DataFrame的merge、filter、groupby和sort算子执行Q5，用来观察专用核函数与算子库之间的差别。

混合版本按Arrow batch把lineitem划成互不重叠的CPU和GPU区间，GPU端异步启动copy路径，主线程同时执行CPU区间，最后按国家合并两个定点数结果。实验使用25%、50%和75%三种CPU比例。这里的“混合”是数据并行，不是把不同关系算子流水化到CPU和GPU。

4 实验设计

实验分为两级。tiny数据只有6条lineitem，用来检查日期、地区、同国家条件和收入公式。正式数据使用TPC-H V3.0.1 dbgen生成的SF1。冻

结矩阵包含专用CPU和Acero各6种线程数、三种CUDA模式、三种混合比例以及cuDF，共19组配置。每组先运行3次预热，再启动10个独立冷进程测量，总计57次预热和190次正式样本。工具保留每次stdout、stderr、退出状态、进程RSS、原始CSV和环境信息，并从原始记录重新计算中位数、p95和标准差。

服务器包含2路AMD EPYC 9654和NVIDIA GeForce RTX 4090，本次使用第0张GPU；软件环境为CUDA 12.6、Arrow/PyArrow 23.0.1和cuDF 26.06.00。正确性要求不同后端输出的国家顺序和 10^4 尺度收入相同，并与官方q5.out的两位小数结果一致。正式证据目录的manifest摘要前12位为e3337842d367，完整值保存在生成文件和证据目录中。

为了缩短课程项目周期，正式实验只做SF1，没有继续做SF10；也没有做查询并发、NUMA绑定和完整Nsight分析，也没有GPU显存峰值采样。因此，本文主要回答“这些实现能否正确运行、主要时间花在哪里”，不尝试给出普遍性能模型。

5 实验结果

5.1 正确性

tiny数据的预期结果为JAPAN 190.00、INDIA 90.00。修正decimal语义后，CPU与三种CUDA模式在 10^4 尺度上的结果哈希一致。

SF1上19组配置的190次正式结果哈希均为542abf4003633c7c。表1列出的五行与官方q5.out逐行一致，核对过程没有把字符串转换为浮点数。

Table 1 SF1 Q5结果与官方答案核对

国家	收入
INDONESIA	55502041.17
VIETNAM	55295087.00
CHINA	53724494.26
INDIA	52035512.00
JAPAN	45410175.70

5.2 性能观察

表2给出3次预热、10次正式运行的中位数。内部查询时间由各后端按统一字段报告，冷进程时间由外部控制器测量。我原先预期GPU或混合执行会在六百万行lineitem上占优，但本次结果没有支持这个预期。专用CPU查询时间最短，cuDF的查询阶段次之；三种手写CUDA和混合路径仍包含较重的计划准备、分配与Arrow列暂存成本。

更细的CUDA计时能说明内存

Table 2 SF1时间中位数 (ms)

后端	内部total	冷进程墙钟
专用CPU (16线程)	61.414	381.285
Arrow Acero (32线程)	321.535	707.231
cuDF	116.427	3682.381
gpu-copy	314.151	1008.970
gpu-managed	358.158	1006.683
gpu-mapped	412.264	1134.574
hybrid (75% CPU)	222.832	888.370

模式差异。gpu-copy的H2D中位数为6.526 ms, kernel为1.347 ms; managed的预取阶段为6.425 ms, kernel为1.345 ms; mapped没有大块显式H2D, 但kernel增加到2.681 ms。这说明mapped并不是取消PCIe访问, 而是把输入传输改成核函数运行期间的远程读取。managed也没有在本次SF1中超过copy。

混合比例从25% CPU增加到75% CPU时, 查询中位数从295.435 ms依次下降到254.478 ms和222.832 ms。75%配置记录到61.460 ms的后端持续时间重叠, 但这只能说明两个后端的运行区间相交。因为没有Nsight时间线, 本文不声称CPU scan和CUDA kernel本身发生了重叠。

两种时间列不能混为同一种排名。query_total_ms用于观察后端查询阶段; process_elapsed_ms还包含进程启动、Arrow校验与加载, cuDF还包含Conda和Python导入。比如cuDF查询中位数为116.427 ms, 冷进程却达到3682.381 ms。只挑其中一列都可能得到片面的结论。

6 讨论与不足

本实验有几处比较明显的不足。第一, 专用C++路径和Arrow/cuDF路径还没有完全统一执行准备口径; 虽然输入都是Arrow, 专用路径、Acero和cuDF的转换工作仍然不同。第

二, GPU只做lineitem扫描, 不是完整的GPU查询计划。第三, 三种CUDA模式每次运行都重新分配和准备内存, 无法代表常驻数据库缓冲池。第四, 只测试SF1, 数据量不足以说明更大规模下的交叉点。第五, 聚合使用全局原子加, 没有实现block内局部归约。第六, 混合路径两侧会重复准备计划, 目前没有证明更细的kernel级重叠。

这些问题不影响本次“实现并比较不同内存模式”的基本目标, 但会限制性能结论。后续最直接的改进是复用CPU计划和GPU缓冲区, 再增加SF10、常驻查询和多查询实验。我认为这些改动比继续微调当前单次kernel更可能改变整体结果。

7 结论

本文完成了一个可以复现的TPC-H Q5 CPU-GPU实验原型。实现表明, GPU内存模式的差别不仅是API差别, 还会改变PCIe传输发生的位置和核函数的数据访问代价。对于当前冷进程SF1查询, 专用CPU的查询中位数最短; copy在三种CUDA模式中最好, mapped虽无显式大块H2D却最慢; 混合执行正确但没有获得加速。这个负结果说明并行执行本身不等于端到端优化, 计划复用和数据准备同样重要。项目还通过官方答案修复了原有decimal截断问题。由于系统仍是固定查询执行器、实验规模也较小, 本文把更大规模、常驻生命周期和完整GPU查询计划留作后续工作。

- [1] Transaction Processing Performance Council. TPC Benchmark H Standard Specification Revision 3.0.1[EB/OL]. <https://www.tpc.org/tpch/>.
- [2] Apache Arrow Project. Apache Arrow Documentation[EB/OL]. <https://arrow.apache.org/docs/>.
- [3] NVIDIA. CUDA C++ Programming Guide[EB/OL]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [4] RAPIDS. cuDF Documentation[EB/OL]. <https://docs.rapids.ai/api/cudf/stable/>.